

# Table of Contents

1. System Architecture
  - What the Desktop App Does
  - The Modules
  - How a Robot Gets Driven
  - The Collision Planner
  - The Board Graph
  - Coordinate Systems
  - The Logger
  - Build System
2. Network Protocols
  - How the Pieces Connect
  - AR App <-> Desktop: TCP/JSON
  - Desktop <-> EV3: Binary Protocol
  - Timing Constants
  - Session Management
3. AR Mobile App
  - What the AR App Does
  - Scenes
  - How the Phone Finds the Desktop
  - Networking Scripts
  - Game Flow
  - AR Integration
  - Input Handling
  - Localization
4. Cross-Component Interactions
  - The Three Components
  - Network Topology
  - Game Startup
  - Robot Discovery
  - Driving a Robot (Desktop-Initiated)
  - Joystick Driving (Phone-Initiated)
  - Shop Interaction
  - Roulette
  - Splatoon Mini-Game
  - Timing Summary
5. Logger Architecture
  - How Components Interact With It
  - Key Interaction Patterns
  - Where Do the Logs Go?

# System Architecture

This document explains how the desktop application is put together and what each part does.

---

## What the Desktop App Does

The desktop app is the central piece of the Mindstorms Kart system. It does three things:

1. **Tracks robots** using a webcam and ArUco markers glued to the robots
2. **Drives robots** to board nodes by planning paths and sending motor commands
3. **Runs the board game** and talks to the AR phones over TCP/JSON

The app is written in C++23 with Qt6 for the GUI, OpenCV for camera tracking, Boost.Asio for networking, and Mosquitto for MQTT.

---

## The Modules

The source code lives in `src/` and is split into these subdirectories:

```
main.cpp
|
+-- communication/    TCP + UDP servers for the AR phones
+-- protocol/         Binary encode/decode for EV3 commands
+-- mqtt_client/      MQTT wrapper (talks to EV3 bricks)
+-- udp_link/         UDP transport (talks to EV3 bricks)
+-- robot_controller/ Per-robot command facade + fleet discovery
+-- robot_tracker/    ArUco marker detection via webcam
+-- pathfinding/       A* path planner on a 2D grid
+-- collision_avoidance/ Multi-robot collision avoidance
+-- path_follower/    Pure pursuit controller (turns paths into motor
commands)
+-- map/              Board graph, MapWindow, driving UI
+-- gameplay/         Game session, turn manager, mini-game runner
+-- player/           Player data (coins, stars, inventory, effects)
+-- items/            Shop items and their effects
+-- mini_games/       Mini-game framework (Splatoon)
+-- qtpanels/         Qt windows and debug panels
+-- camera/           Camera calibration helper
+-- utilities/        Logger, enum helpers
```

---

## How a Robot Gets Driven

This is the most important thing the desktop app does. Here is what happens step by step when a user clicks a node on the board:

1. User clicks a node on the board (MapWindow)
2. MapWindow asks the collision planner to route the robot there  
`collision_planner::request_route_to(robot_id, target_node)`
  - Finds the shortest path along the board's node graph (Dijkstra)

- Builds a drivable polyline that avoids parked robots (A\*)
  - Rounds corners so the robot doesn't make sharp turns
  - Publishes the path as a path\_update
3. Every 33ms the MapWindow runs a tick:
    - Pulls the robot's current effective path from the planner
    - Feeds it to the path\_follower along with the robot's current pose
    - The path\_follower computes a velocity command (speed + turn rate)
    - Converts it to brick percentages (-100..100)
    - Calls robot\_controller::set\_drive(speed\_pct, turn\_pct)
  4. The robot\_controller encodes a binary UDP datagram:  
[nn][0x01][speed][turn] (4 bytes, little-endian)  
and sends it to the EV3 brick on port 7778
  5. The EV3 brick runs the motors
  6. The EV3 sends heartbeats back on port 7779
    - robot\_fleet picks them up and creates/refreshes robot\_controller instances
    - robot\_controller updates state and battery info
  7. The webcam detects the ArUco marker on the robot at ~30 FPS
    - marker\_tracker returns pixel positions
    - MapWindow converts them to board coordinates via a homography matrix
    - Feeds the new poses into the collision planner
    - The planner may re-route if another robot is in the way (CTFS)
  8. This loop continues until the robot reaches its target node
- 

## The Collision Planner

When multiple robots move at the same time, they might collide. The collision planner (collision\_avoidance/collision\_planner.hpp) handles this.

The algorithm is called **CTFS** (Component-Token Freeze-Senior Step-Off). In plain terms:

1. Every tick, the planner looks at which robots are close to each other
2. It groups conflicting robots into connected components
3. In each group, the robot with the **lowest ID** keeps moving
4. The other robots either:
  - **Freeze** (stop in place and wait)
  - **Step off** (drive to a siding out of the way, then resume after the other passes)
  - **Park beside** the target node if it is already occupied (first-come-first-served)

The planner publishes versioned path\_update snapshots. Each update contains a version number, a list of waypoints, and a state (nominal / avoiding / holding). The path follower reads these and drives accordingly.

---

## The Board Graph

The physical game board is a 300 cm x 185 cm surface with 21 nodes arranged in a 4-column x 5-row grid. Each node has: - A position (screen pixels for the Qt window, centimetres for the planner) - Neighbour nodes (undirected edges for movement) - Tile effects (Plus, Minus, Shop, Hazard)

The board graph is defined once in `Board::Board()` and never changes during the game. It can be broadcast to the AR phones over TCP when they ask for it.

---

## Coordinate Systems

The app works in three coordinate systems:

| System        | Units           | Used For                                    |
|---------------|-----------------|---------------------------------------------|
| Camera pixels | px              | Raw ArUco marker detection                  |
| Board frame   | cm (300 x 185)  | Path planning, collision avoidance, driving |
| Screen pixels | px (1400 x 860) | Rendering the board, mouse click handling   |

Conversions: - **Camera** -> **Board**: A homography matrix built from 6 calibration markers (IDs 5-10) using `cv::solvePnP`. Built once when at least 4 markers are visible. - **Board** -> **Screen**: Simple scaling: 1400/300 horizontally, 860/185 vertically.

The path planner and followers work entirely in board centimetres. Only the rendering uses screen pixels.

---

## The Logger

Every module logs through a central Logger singleton (`utilities/Logger.hpp`). It works like this:

1. Any code writes `LOG(LogLevel::Info) << "something happened";`
2. A `LogStream` object is created, captures the message text
3. When the stream goes out of scope, it pushes the string into a thread-safe queue
4. A background thread drains the queue and appends everything to `logs.log`
5. Most messages are also printed to the console, except ENTRY/EXIT trace markers (to keep the console clean)

The logger never blocks the calling thread. This is important because the robot control loop needs to run at a steady rate.

```
// Simple logging
LOG(LogLevel::Info) << "Player " << id << " joined";

// Automatic function timing (logs ENTRY + EXIT with elapsed time)
void myFunction() {
```

```

    LOG_ENTRY_EXIT;
    // ...
}

// Set which severity levels are active
SET_LOG_LEVELS(LogLevel::Info, LogLevel::Warn, LogLevel::Error);

```

---

## Build System

The project uses CMake with vcpkg for dependencies:

| Dependency                   | What it provides                       |
|------------------------------|----------------------------------------|
| Qt6 (Core, Widgets, Network) | GUI, event loop, signals/slots         |
| OpenCV                       | Camera capture, ArUco marker detection |
| nlohmann_json                | JSON parsing and serialization         |
| Boost.Asio                   | TCP and UDP async networking           |
| libmosquitto                 | MQTT client for talking to EV3 bricks  |

```

# Configure
cmake --preset <preset>

# Build
cmake --build build

# Build docs
cmake --build build --target docs

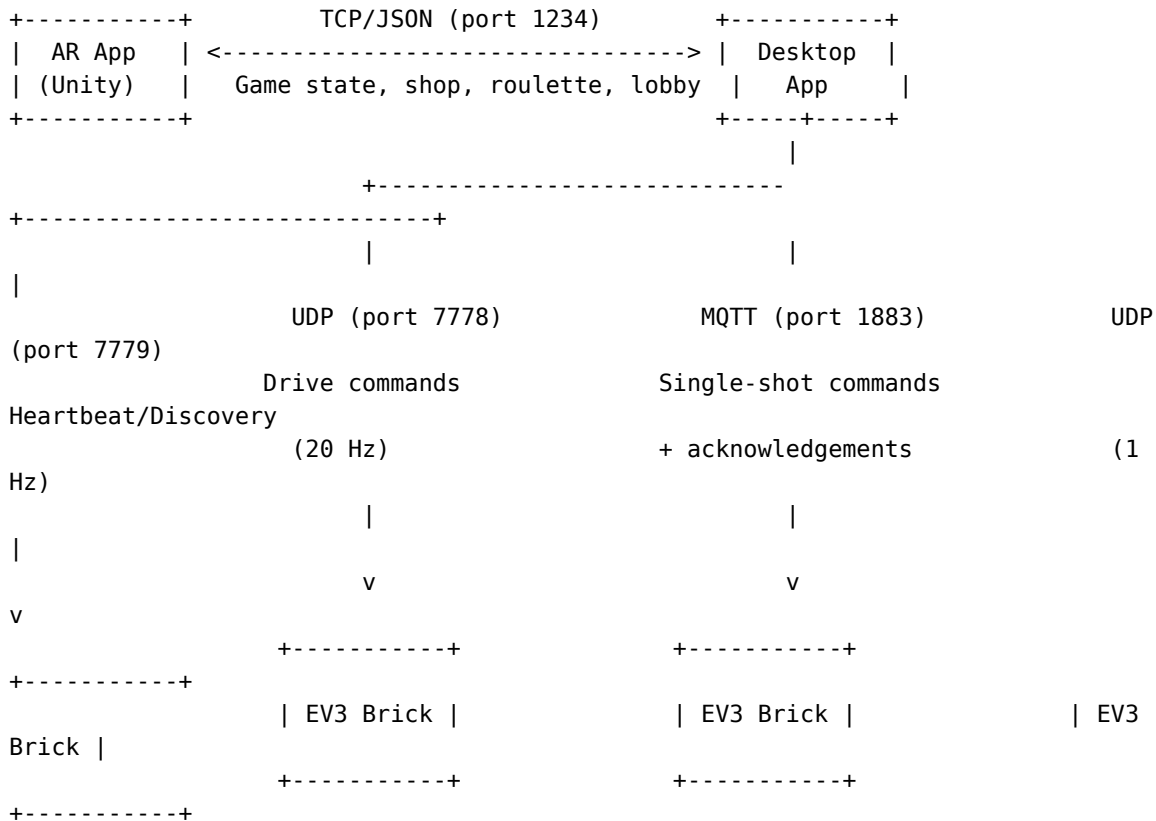
```

## Network Protocols

The three components (Desktop, AR phones, EV3 robots) talk to each other over three different network connections. Each one was chosen for a specific reason.

---

### How the Pieces Connect



- **TCP/JSON on port 1234** connects the AR phones to the desktop app. Used for game events: opening shops, placing bets, sending board data, and sending commands like `play_sound`.
  - **UDP on port 7778** goes from the desktop to the EV3 robots. Used for the continuous drive loop (20 times per second).
  - **MQTT on port 1883** goes between the desktop and the EV3 robots. Used for one-off commands that must not be lost, like `stop`, `beep`, or `drive_for`.
  - **UDP on port 7779** goes from the EV3 robots to everyone on the network (broadcast). Used for heartbeats so the desktop knows which robots are alive.
- 

### AR App <-> Desktop: TCP/JSON

The AR phone connects to the desktop over TCP on port 1234. Every message is a JSON object followed by a newline. The desktop reads these in a background thread and routes each one based on its "type" field.

## How Routing Works

The desktop has a map of message types to handler functions. When it receives a message, it looks up the "type" string and calls the matching handler. Adding a new message type is one line:

```
register_message_handler("my_new_type",
    [](const nlohmann::json& msg, uint8_t from_id) {
        auto payload = msg.get<ns::MyEvent>();
        // handle it...
    });
```

All handlers are registered before the server starts, so there is no locking needed during dispatch.

## The Message Types

| Type             | Direction     | What It Does                  | Payload                                                  |
|------------------|---------------|-------------------------------|----------------------------------------------------------|
| "open_shop"      | AR -> Desktop | Player wants to open the shop | {playerId: int}                                          |
| "close_shop"     | AR -> Desktop | Player closes the shop        | {playerId: int}                                          |
| "selected_item"  | AR -> Desktop | Player selects an item        | {playerId: int, itemID: int}                             |
| "roulette_start" | AR -> Desktop | Player starts roulette        | {playerId: int}                                          |
| "make_bet"       | AR -> Desktop | Player places a bet           | {playerId: int, amount: int, choice: int, bet_type: int} |
| "end_roulette"   | AR -> Desktop | Player leaves roulette        | {playerId: int}                                          |
| "pong"           | AR -> Desktop | Response to keep-alive ping   | {}                                                       |
| "board_load"     | AR -> Desktop | Request board graph           | {}                                                       |
| "board_load"     | Desktop -> AR | Full board graph              | Array of 21 nodes with neighbours and tile effects       |
| "play_sound"     | Desktop -> AR | Play a sound on the phone     | {audioId: int, vol: int}                                 |

## JSON DTOs

All the message structures are defined in `communication/network_messages.hpp` using `nlohmann_json`'s automatic serialization:

```
namespace ns {
    struct OpenShopEvent { int playerId; };
    struct SelectedItemEvent { int playerId, itemID; };
    struct CloseShopEvent { int playerId; };
    struct StartRouletteEvent { int playerId; };
    struct GetPlayerBetEvent { int playerId, amount, choice; Bet_type
bet_type; };
}
```

```

    struct EndRouletteEvent { int playerId; };
}

```

---

## Desktop <-> EV3: Binary Protocol

The full protocol specification lives in `shared/protocol.md`. Here is a summary of what matters on the desktop side.

### Why Two Transports?

The desktop talks to the EV3 in two ways:

- **UDP (port 7778)** is used for the drive loop. Drive commands are sent 20 times per second. If one gets lost, the next one will arrive 50ms later and fix it. No need for reliability.
- **MQTT (port 1883)** is used for commands like `stop`, `beep`, and `drive_for`. These must arrive reliably, so MQTT's QoS 1 (at-least-once delivery) is used.

### Binary Wire Format

All numbers are little-endian. The codec is in `protocol/codec.hpp`.

### Commands (Desktop -> EV3)

**UDP datagrams** (drive loop):

| Opcode | Name        | Params                                            | Wire Format                       |
|--------|-------------|---------------------------------------------------|-----------------------------------|
| 0x01   | drive       | speed_pct<br>(-100..100),<br>turn_pct (-100..100) | [nn][0x01]<br>[speed:u8][turn:u8] |
| 0x02   | wheel_drive | left (-100..100),<br>right (-100..100)            | [nn][0x02]<br>[left:u8][right:u8] |

The nn byte is the robot's index (e.g., 1 for `ev3_01`).

**MQTT payloads** (one-off commands):

| Opcode | Name      | Params                       | Wire Format                             |
|--------|-----------|------------------------------|-----------------------------------------|
| 0x03   | drive_for | distance_mm,<br>speed_mm_s   | [0x03][id:u16][dist:i32]<br>[speed:i16] |
| 0x04   | turn      | angle_deg,<br>rate_dps       | [0x04][id:u16][angle:i16]<br>[rate:i16] |
| 0x05   | stop      | brake (0 or 1)               | [0x05][id:u16][brake:u8]                |
| 0x06   | beep      | duration_ms,<br>frequency_hz | [0x06][id:u16][dur:u16]<br>[freq:u16]   |
| 0x07   | ping      | (none)                       | [0x07][id:u16]                          |



### Status (EV3 -> Desktop)

**MQTT ack** (topic mind/<robot\_id>/status):

[0x10][cmd\_id:u16][state:u8][battery\_mv:u16][err\_code:u8]

| Field      | What It Is                                                                   |
|------------|------------------------------------------------------------------------------|
| cmd_id     | Which command this is acknowledging (or 0xFFFF if none)                      |
| state      | 0=idle, 1=driving, 2=moving, 3=error                                         |
| battery_mv | Battery voltage in millivolts                                                |
| err_code   | 0=ok, 1=unknown opcode, 2=bad length, 3=bad param, 4=motor fault, 5=internal |

**UDP heartbeat** (broadcast every 1 second):

[0x11][nn:u8][state:u8][battery\_mv:u16]

The heartbeat is broadcast to 255.255.255.255:7779. The source IP of the datagram tells the desktop which EV3 sent it. This is how robots get discovered automatically.

### MQTT Topics

| Topic                  | Direction       | What Goes There             |
|------------------------|-----------------|-----------------------------|
| mind/<robot_id>/cmd    | Desktop -> EV3  | One-off commands            |
| mind/<robot_id>/status | EV3 -> Desktop  | Acknowledgements            |
| mind/+ /status         | Desktop (sub)   | Wildcard for all robot acks |
| mind/anchor            | Desktop -> MQTT | Camera anchor pose (JSON)   |
| mind/robots/position   | Desktop -> MQTT | Robot positions (JSON)      |

<robot\_id> is ev3\_NN where NN matches the UDP robot index.

## Timing Constants

| Constant            | Value    | What It Means                                                  |
|---------------------|----------|----------------------------------------------------------------|
| PUBLISH_MS          | 50 ms    | How often the desktop sends drive commands (20 Hz)             |
| WATCHDOG_MS         | 150 ms   | EV3 auto-stops if no drive command arrives for 150ms           |
| HEARTBEAT_MS        | 1000 ms  | How often EV3 broadcasts its heartbeat                         |
| OFFLINE_MS          | 3000 ms  | Desktop marks a robot as offline if no heartbeat for 3 seconds |
| MOVE_ACK_TIMEOUT_MS | 30000 ms | move_relative abort timeout                                    |
| RELEASE_IDLE_MS     | 100 ms   | Joystick released after 100ms of (0,0) input                   |

## EV3 Calibration (on-brick, not on wire)

| Constant           | Value | What It Means                            |
|--------------------|-------|------------------------------------------|
| WHEEL_MAX_MM_S     | 450   | Speed at 100% throttle (mm/s)            |
| DRIVE_MAX_TURN_DPS | 250   | Turn rate at 100% steer (deg/s)          |
| WHEEL_DIAMETER_MM  | 56    | For converting motor degrees to distance |
| AXLE_TRACK_MM      | 126   | Distance between wheels                  |

## Session Management

The SessionRegistry (communication/session\_registry.hpp) keeps track of which AR phones are connected. Each phone gets a session when it first connects over TCP:

```
struct PlayerSession {
    ConnectionID id;
    std::string device_name;
    std::shared_ptr<TcpConnection> tcp_conn;
    boost::asio::ip::udp::udp_endpoint udp_endpoint;
    std::chrono::steady_clock::time_point last_seen;
    std::uint64_t bytes_tx, bytes_rx;
    std::unordered_map<std::string, std::pair<uint64_t, uint64_t>> typed_traffic;
};
```

- create\_session(name, conn) - Assigns a connection ID on first TCP connect
- update\_udp\_endpoint(id, ep) - Links a UDP endpoint to an existing TCP session
- touch\_session(id) - Updates the last-seen timestamp
- sweep\_stale\_sessions(timeout) - Removes phones that have been silent for too long
- record\_traffic(id, tx, rx, category) - Tracks bytes sent/received per message type

The registry uses a `std::shared_mutex` so that `dispatch` (which happens a lot) can run concurrently, while `connect/disconnect` (which is rare) takes a write lock.

## AR Mobile App

The AR app is a Unity project that runs on Android phones. It is the player's controller: it shows the board, handles touch input, runs mini-games, and talks to the desktop app over TCP.

---

### What the AR App Does

Each player has a phone running this app. The app does four things:

1. **Shows the game board** using the phone's camera (AR)
2. **Sends player actions** to the desktop app (open shop, place bets, drive joystick)
3. **Receives game state** from the desktop app (board updates, score, sounds)
4. **Runs mini-games** (Splatoon) that use the camera to paint territory

The app is written in C# with Unity 6, ARFoundation, and the Input System package.

---

### Scenes

The app has four scenes:

| Scene                | What Happens There                                                                      |
|----------------------|-----------------------------------------------------------------------------------------|
| <b>Startscreen</b>   | Main menu. Player enters their name and chooses to create or join a game.               |
| <b>Lobby</b>         | Shows who is connected. The host sees a "Start Game" button when enough players are in. |
| <b>GameScene</b>     | The board game. Shows the AR board, handles dice, shop, roulette, and driving.          |
| <b>SplatoonScene</b> | Mini-game. Players paint territory on a grid by tapping their phone.                    |

Scene transitions are managed by Unity's `SceneManager.LoadScene()`. The `LobbyManager` singleton persists across scene loads.

---

### How the Phone Finds the Desktop

The phone does not know the desktop's IP address ahead of time. Instead:

1. The phone broadcasts a UDP message on port 9999: `UDPServerAddress,1,IP`
2. The desktop receives this and replies with its own IP: `SERVEREV3,<desktop_ip>,IP`
3. The phone stores the desktop IP in `NetworkRegistry.ServerIp` and starts connecting over TCP

This happens automatically in `UdpListener.cs`. The `NetworkRegistry` singleton (`DontDestroyOnLoad`) keeps the IP available across all scenes.

Similarly, the desktop sends EV3 robot IPs it discovers: `MindstormIP,<ev3_ip>,IP`. The phone stores these in `NetworkRegistry.Ev3Ip`.

---

## Networking Scripts

| Script                           | What It Does                                                                    |
|----------------------------------|---------------------------------------------------------------------------------|
| <code>TCPClient.cs</code>        | Connects to the desktop's TCP server on port 1234. Runs a background read loop. |
| <code>TCPSender.cs</code>        | Sends JSON messages to the desktop. Handles player registration.                |
| <code>TCP_Handler.cs</code>      | Routes incoming TCP messages to the right handler (game events, lobby, etc.).   |
| <code>UdpListener.cs</code>      | Listens on port 9999 for desktop/robot IP discovery broadcasts.                 |
| <code>UDPLobbySender.cs</code>   | Sends lobby messages to the desktop over UDP (dual-transport with TCP).         |
| <code>Ev3UdpClient.cs</code>     | Sends joystick drive commands directly to EV3 robots over UDP (port 7778).      |
| <code>MqttRobotClient.cs</code>  | Connects to the EV3 MQTT broker for joystick commands (mind/<id>/cmd).          |
| <code>MessageForwarder.cs</code> | Bridges background-thread TCP messages to Unity's main thread.                  |
| <code>NetworkRegistry.cs</code>  | Singleton that stores discovered server and EV3 IPs. Available in all scenes.   |

---

## Game Flow

### 1. Startscreen

The player sees the main menu. They can: - Enter their player name - Choose the number of players and rounds - Tap "Create Game" (becomes host) or "Join Game"

When the host taps "Create Game", the app sends `create_host` to the desktop. The desktop creates a session, assigns a player ID (always 1 for the host), and sends back `host_created`. The host's phone then loads the Lobby scene.

When a guest taps "Join Game", the app sends `join_game`. The desktop assigns the next available player ID and sends `player_assigned`. The guest's phone loads the Lobby scene.

### 2. Lobby

The lobby shows a list of connected players. The host sees a "Start Game" button that only appears when enough players have joined.

The lobby requests a player list from the desktop using both UDP and TCP (dual transport for reliability). The desktop replies with `player_list` containing all connected player names and the expected count.

When the host taps “Start Game”, it sends `start_game`. The desktop broadcasts `game_started` to all phones, and everyone loads the `GameScene`.

### 3. `GameScene` (Board Game)

This is the main game. The phone shows the board using AR (camera feed). The desktop drives the robots, manages turns, and sends updates.

Key interactions: - **Board load**: The phone requests the board graph from the desktop via TCP (`board_load`). The desktop sends back all 21 nodes with neighbours and tile effects. - **Shop**: The desktop sends `open_shop` when a player lands on a shop tile. The phone shows the shop UI. The player picks an item, and the phone sends `selected_item` back. - **Roulette**: The desktop sends `roulette_start` when a player lands on a roulette tile. The phone shows the betting UI. The player places a bet via `make_bet`. - **Dice**: The desktop sends `play_sound` to all phones when dice are rolled. - **Driving**: The joystick on the phone sends drive commands directly to the EV3 robot via UDP and MQTT (not through the desktop).

### 4. `SplattoonScene` (Mini-Game)

The Splattoon mini-game is a territory-painting game. Players tap cells on a grid to claim them. The grid is rendered as a transparent overlay on the camera feed.

Key scripts: - `FieldGrid.cs` - Creates and manages the grid of claimable cells - `CellClaimSender.cs` - Sends cell claims to the desktop over TCP - `GridReceiver.cs` - Receives grid updates from other players via MQTT - `SimpleJoystick.cs` - Drives the robot to the selected cell - `ScorePanel_Splattoon.cs` - Shows the score bar at the top - `InklingFollower.cs` - AR robot visual that follows the EV3’s position

The Splattoon mini-game talks to EV3 robots via MQTT for position updates and joystick control.

---

## AR Integration

The app uses `ARFoundation` for camera-based augmented reality:

- **`ArucoAnchor.cs`** - Detects `ArUco` markers in the camera feed and uses them to anchor the virtual board to the physical board. This is how the phone knows where the board is.
- **`ARImageDistanceManager.cs`** - Calculates the distance from the phone to detected markers. Used to position the AR robot visual at the right depth.
- **`AnchorTestVisualizer.cs`** - Debug tool for visualizing anchor positions.

The AR system is not required for the game to work. The app can run without AR for testing purposes (the `GameManager` handles this gracefully).

---

## Input Handling

- **InputHandler.cs** - Central input handler. Connects to the joystick and forwards input to the appropriate subsystem.
  - **Joystick.cs** - Virtual joystick UI element. Players drag to steer the robot.
  - **PlayerIdSelector.cs** - UI for selecting which robot to control (used in Splatoon scene).
- 

## Localization

The app supports multiple languages:

- **LanguageManager.cs** - Manages the current language
  - **LanguageDropdown.cs** - UI dropdown for language selection
  - **LocalizedText.cs** - Attach to any TextMeshPro component to auto-translate
  - **LocalizationDatabase.cs** - Stores translation strings
-

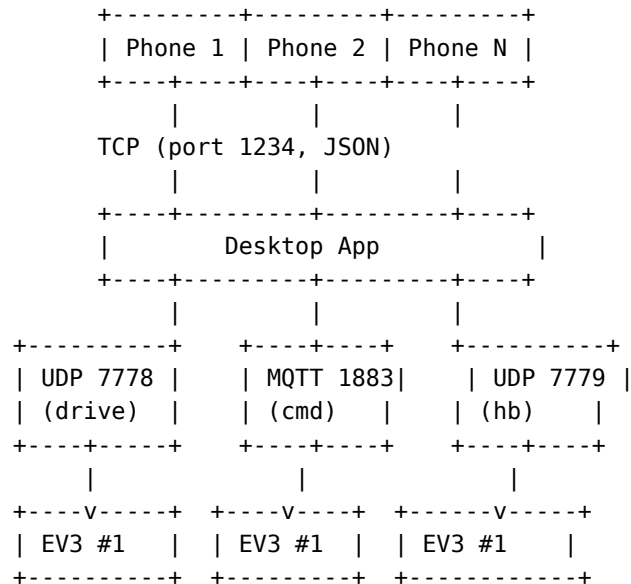
## Cross-Component Interactions

This document explains how the three components (Desktop, AR phones, EV3 robots) work together during a game. It walks through the main scenarios step by step.

### The Three Components

| Component   | Technology             | Runs On                         |
|-------------|------------------------|---------------------------------|
| Desktop App | C++/Qt/OpenCV          | Any desktop OS                  |
| AR App      | Unity/C#               | Android phones (one per player) |
| EV3 Robots  | MicroPython (PyBricks) | LEGO Mindstorms EV3 bricks      |

### Network Topology



There are three kinds of connections:

1. **Phones** <-> **Desktop** over TCP (JSON messages)
2. **Desktop** -> **Robots** over UDP (drive commands) and MQTT (reliable commands)
3. **Robots** -> **Desktop** over UDP broadcast (heartbeats)

The phones also talk directly to the robots for joystick driving (UDP + MQTT), bypassing the desktop.

### Scenario 1: Game Startup

This is what happens from the moment someone opens the app until the game starts.



1. Phone A opens the app
    - UdpListener broadcasts "UDPServerAddress,1,IP" on port 9999
    - Desktop receives it, replies with "SERVEREV3,<desktop\_ip>,IP"
    - Phone A stores the desktop IP in NetworkRegistry
  2. Phone A taps "Create Game"
    - TCPSender sends {"type":"create\_host","id":1} to desktop
    - Desktop creates a session, assigns player ID 1
    - Desktop replies with {"type":"host\_created","playerId":1}
    - Phone A loads the Lobby scene
  3. Phone B opens the app
    - UdpListener broadcasts, gets desktop IP
    - Phone B taps "Join Game"
    - TCPSender sends {"type":"join\_game","id":2} to desktop
    - Desktop assigns player ID 2
    - Desktop replies with {"type":"player\_assigned","playerId":2}
    - Phone B loads the Lobby scene
  4. Both phones are in the Lobby
    - Both phones request the player list ("request\_player\_list")
    - Desktop replies with {"type":"player\_list","players":["Player 1","Player 2"]}
    - Phone A (host) sees the "Start Game" button
  5. Host taps "Start Game"
    - Phone A sends {"type":"start\_game","id":1}
    - Desktop broadcasts {"type":"game\_started"} to all phones
    - All phones load the GameScene (board)
- 

## Scenario 2: Robot Discovery

When the desktop app starts, it needs to find the EV3 robots on the network.

1. EV3 robot boots up
  - Runs the PyBricks program
  - Every 1 second, broadcasts a UDP heartbeat to 255.255.255.255:7779
  - Heartbeat: [0x11][nn][state][battery\_mv:u16]
2. Desktop receives the heartbeat
  - robot\_fleet picks up the source IP from the datagram
  - Creates a robot\_controller for this robot
  - Sends "MindstormIP,<ev3\_ip>,IP" to all phones on port 9999
  - Phones store the EV3 IP in NetworkRegistry
3. Desktop registers with the robot
  - Sends "SERVEREV3" to the robot on UDP port 7780
  - Robot now knows where to send heartbeats

4. MQTT connection
    - Desktop subscribes to "mind+/status" for all robot acks
    - Robot connects to the desktop's MQTT broker
    - Bidirectional MQTT is now ready
- 

### **Scenario 3: Driving a Robot (Desktop-Initiated)**

When a player clicks a node on the board in the desktop app.

1. User clicks node 15 on the board
    - MapWindow detects the click
    - Asks collision\_planner to route robot #1 there
  2. Path planning
    - collision\_planner finds shortest path on the board graph (Dijkstra)
    - Builds a drivable polyline (A\* on grid, 5cm resolution)
    - Rounds corners (50cm radius) so the robot doesn't jerk
    - Publishes path\_update with version, waypoints, and state
  3. Path following (every 33ms)
    - path\_follower reads the robot's current pose from robot\_tracker
    - Computes cross-track error and heading error
    - Pure pursuit controller decides: speed and turn rate
    - Converts to brick percentages (-100..100)
    - Calls robot\_controller::set\_drive(speed\_pct, turn\_pct)
  4. Command encoding
    - robot\_controller encodes: [nn][0x01][speed][turn] (4 bytes)
    - Sends UDP datagram to the EV3 on port 7778
  5. EV3 executes
    - Receives the datagram
    - Sets motor speeds using the DriveBase
    - Starts watchdog timer (150ms) - if no new command arrives, stops
  6. Continuous loop
    - Steps 3-5 repeat every 50ms (20 Hz)
    - robot\_tracker updates the robot's pose from the webcam
    - collision\_planner re-routes if another robot is in the way
    - Loop ends when the robot reaches its target node
- 

### **Scenario 4: Joystick Driving (Phone-Initiated)**

When a player uses the on-screen joystick in the AR app. This bypasses the desktop entirely.

1. Player drags the joystick on their phone
  - InputHandler reads the joystick values (x, y from -1 to 1)
  - Converts to speed and turn percentages
2. Commands sent directly to the EV3

- Ev3UdpClient sends drive commands: [nn][0x01][speed][turn]
  - MqttRobotClient sends {"type":"cmd","opcode":"drive","params":[speed,turn]}
  - Both go directly to the EV3, not through the desktop
3. EV3 executes
    - Same as desktop-initiated driving
    - Motors run, watchdog timer active
  4. When joystick is released
    - InputHandler detects (0,0) for 100ms
    - Sends stop command to the EV3
    - EV3 brakes
- 

## Scenario 5: Shop Interaction

When a robot lands on a shop tile.

1. Desktop detects the robot reached a shop node
    - turn\_manager checks the tile effect
    - Sends "open\_shop" to the player's phone via TCP
  2. Phone shows shop UI
    - ShopImporter receives the message
    - Displays available items (Boo, Shell, Golden Shoes, etc.)
    - Player taps an item
  3. Phone sends selection back
    - TCPSender sends {"type":"selected\_item","playerId":1,"itemID":3}
    - Desktop receives it
  4. Desktop processes the purchase
    - Checks if the player has enough coins
    - Deducts coins, adds item to inventory
    - Sends confirmation or rejection back to the phone
- 

## Scenario 6: Roulette

When a robot lands on a roulette tile.

1. Desktop detects roulette tile
  - Sends "roulette\_start" to the player's phone
2. Phone shows betting UI
  - RouletteImporter displays the betting options
  - Player chooses: Red/Black/Green, and an amount
  - Player taps "Place Bet"
3. Phone sends the bet
  - TCPSender sends {"type":"make\_bet","playerId":1,"amount":5,"choice":1,"bet\_type":0}

- Desktop receives it
4. Desktop runs the roulette
    - Generates a random result
    - Calculates winnings
    - Sends the result back to all phones
    - Updates scores
- 

## Scenario 7: Splatoon Mini-Game

The Splatoon mini-game is a self-contained mode where players paint territory.

1. Desktop starts the mini-game
    - Sends "start\_minigame" to all phones
    - Phones load the SplatoonScene
  2. Each phone shows the grid
    - FieldGrid creates an NxN grid of cells
    - Cells are overlaid on the camera feed using AR
  3. Player taps a cell
    - CellClaimSender sends {"type": "claim\_cell", "playerId": 1, "x": 3, "y": 5}
    - Desktop receives it
  4. Desktop validates and broadcasts
    - Checks if the cell is already claimed
    - If valid, marks it as claimed by this player
    - Broadcasts the updated grid to all phones via MQTT
    - GridReceiver updates the local grid
  5. Player drives their robot to paint
    - SimpleJoystick sends drive commands to the EV3 via UDP/MQTT
    - InklingFollower follows the EV3's real position in AR
    - As the robot moves, nearby cells get painted
  6. Mini-game ends
    - Desktop sends "end\_minigame" after the timer runs out
    - ScorePanel\_Splatoon shows final scores
    - Phones return to the board game
-

## Timing Summary

| What                 | How Often          | Who Sends               |
|----------------------|--------------------|-------------------------|
| Drive commands (UDP) | 20 Hz (50ms)       | Desktop or Phone        |
| Heartbeats (UDP)     | 1 Hz (1000ms)      | EV3                     |
| Robot discovery      | On heartbeat       | Desktop -> Phone        |
| TCP game events      | On demand          | Phone <-> Desktop       |
| MQTT commands        | On demand          | Desktop or Phone -> EV3 |
| MQTT acks            | After each command | EV3 -> Desktop          |
| Joystick release     | After 100ms idle   | Phone -> EV3            |

## Logger Architecture

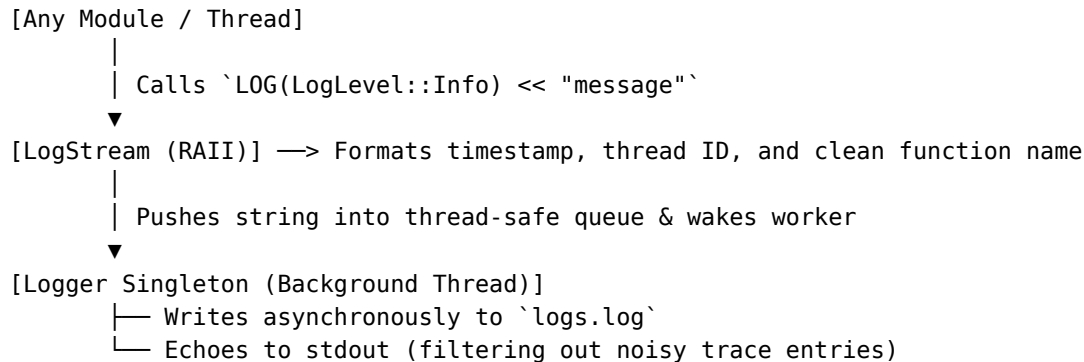
The **Logger** is a centralized, thread-safe diagnostic hub used across all modules in the Mindstorms Kart application (networking, collision avoidance, gameplay, and pathfinding).

Because the application handles real-time robot control and networking where latency is critical, the logger is engineered to **never block the main execution threads**.

---

### How Components Interact With It

Any module (e.g., `TCP_Server.cpp`, `collision_planner.cpp`, or `game_manager.cpp`) can emit logs without worrying about file I/O locks, performance hitches, or thread safety.



### Key Interaction Patterns

#### 1. The Clean Macro Interface (LOG)

Instead of managing logger instances or string formatting manually, any part of the codebase uses a simple streaming macro:

```
LOG(LogLevel::Info) << "Player " << player_id << " joined the lobby.";
```

- **What happens behind the scenes:** A temporary `LogStream` object captures the metadata (severity, thread ID, function name) and accumulates the text via standard stream operators (`<<`). When the statement ends, the destructor safely hands the completed string over to the logger.

#### 2. Asynchronous Queue & Background Worker

- **Non-blocking execution:** When `LOG(...)` finishes, it pushes the string into an internal queue (`std::queue`) protected by a mutex (`std::mutex`) and immediately returns control to the calling thread.
- **Background draining:** A dedicated worker thread (`std::thread`) waits on a condition variable (`std::condition_variable`). When new logs arrive, it wakes up, drains the queue, and appends the messages to `logs.log` in bulk. This ensures heavy logging never stutters the robot controller or network loops.

### 3. Automatic Scope Tracing (LOG\_ENTRY\_EXIT)

To debug complex execution flows or measure function runtime without manual start/stop timers, developers place a single macro at the top of a function:

```
void CollisionPlanner::update() {  
    LOG_ENTRY_EXIT;  
    // ... function logic ...  
}
```

- **How it works via RAII:**

- On construction, it logs an ENTRY marker and starts a high-resolution clock timer (`std::chrono::high_resolution_clock`).
- On destruction (when the function exits and goes out of scope), it automatically computes elapsed wall-clock time and logs: EXIT - 0.042s.
- **Console Filtering:** To keep developer terminals clean, ENTRY and EXIT statements are automatically suppressed from stdout while still being safely recorded in `logs.log`.

### 4. Global Severity Filtering

Teams can control verbosity dynamically across the application using `SET_LOG_LEVELS(...)`:

```
SET_LOG_LEVELS(LogLevel::Error, LogLevel::Warning);
```

- If a module attempts to log at a level that isn't currently active (e.g., `LogLevel::Trace`), the logger drops the message instantly before spending CPU cycles formatting strings or queueing data.

---

## Where Do the Logs Go?

- **logs.log (Root Folder):** The permanent file record containing every single log entry, function entry/exit trace, and error across the entire application's session.

